

Introduction to RNN

Contents

(Vanilla RNN)

LSTM

Introduction

In reality, we might not only receive a sequence as an input and the output is expected to continue a sequence for a machine learning problem. For instance, the task could be a time series problem to predict the stock market, or a natural language problem to predict the next sentence of your email draft. Unlike other regression and classification machine learning problems which do not consider the sequence as a whole, `Sequence Models` need to consider and interpret the **sequential information** of the data, otherwise a great deal of information is lost.

Vanilla RNN

Recurrent Neural Networks (RNNs) are designed to better handle sequential information. RNNs introduce **state variables** to store past information, together with the current inputs, to determine the current outputs. We will go into the details of how RNNs work and how to use them to solve sequence problems.

Neural Networks without Hidden States

Let's recall the structure of an MLP (ANN) first. Let's consider an MLP with a single hidden layer. The hidden layer's activation function is ϕ , and the input is a mini-batch of observations $\mathbf{X} \in \mathbb{R}^{n \times d}$ with batch size n and d inputs. If we use h hidden units in the hidden layer, then the hidden layer's output $\mathbf{H}$ is calculated as:

$$\mathbf{H} = \phi(\mathbf{X}\mathbf{W}_{xh} + \mathbf{b}_h).$$

What is the dimension of $\mathbf{H}$?

$n \times h$

Next, the hidden variable $\mathbf{H}$ is used as the input of the output layer. Suppose the output dimension is o , the output layer is given by

$$\mathbf{O} = \mathbf{H}\mathbf{W}_{ho} + \mathbf{b}_o,$$

Sequence Model

If we use a single layer MLP, we can only model $f(x_t | x_{t-1}, \dots, x_{t-n+1})$, where n is the number of past data we use in the inputs. The disadvantage is that we couldn't use the information prior to time step $t - n + 1$, which could be useful. As a result, in order to address this issue, we propose using a latent variable to model past information, which can be represented as:

$$f(x_t | x_{t-1}, \dots, x_1) \approx f(x_t | h_{t-1}),$$

where h_{t-1} is a *hidden state* (also known as a hidden variable) that stores the sequence information up to time step $t - 1$.

In general, the hidden state at any time step t could be computed based on both the current input x_t and the previous hidden state h_{t-1} :

$$h_t = g(x_t, h_{t-1}).$$

Recurrent Neural Networks with Hidden States

Denote by $\mathbf{H}_t \in \mathbb{R}^{n \times h}$ the hidden variable of time step t , a new weight parameter matrix $\mathbf{W}_{hh} \in \mathbb{R}^{h \times h}$ is introduced. The calculation of the current time step's hidden variable is decided by the input of the current time step combined with the hidden variable of the previous time step:

$$\mathbf{H}_t = \phi(\mathbf{X}_t \mathbf{W}_{xh} + \mathbf{H}_{t-1} \mathbf{W}_{hh} + \mathbf{b}_h).$$

$\mathbf{H}_t$ and $\mathbf{H}_{t-1}$ capture and retain the sequence's historical information up to their current time step. Therefore, such a hidden variable is called a **hidden state**.

$$h_t = g(x_t, h_{t-1}).$$

Recurrent Neural Networks with Hidden States

Notice here that $\mathbf{b}_h$ is a vector, here we perform elementwise operations by invoking the **broadcasting mechanism**. This mechanism works in the following way: First, expand one or both arrays by copying elements appropriately so that after this transformation, the two tensors have the same shape. Second, carry out the elementwise operations on the resulting arrays.

```
x1 <- matrix(1:4, ncol = 2)
x2 <- c(1:2)
x1
##      [,1] [,2]
## [1,]    1    3
## [2,]    2    4
x2
## [1] 1 2
x1 + x2
##      [,1] [,2]
## [1,]    2    4
## [2,]    4    6
```

Recurrent Neural Networks with Hidden States

For time step t , the output of the output layer is similar to the computation in the MLP:

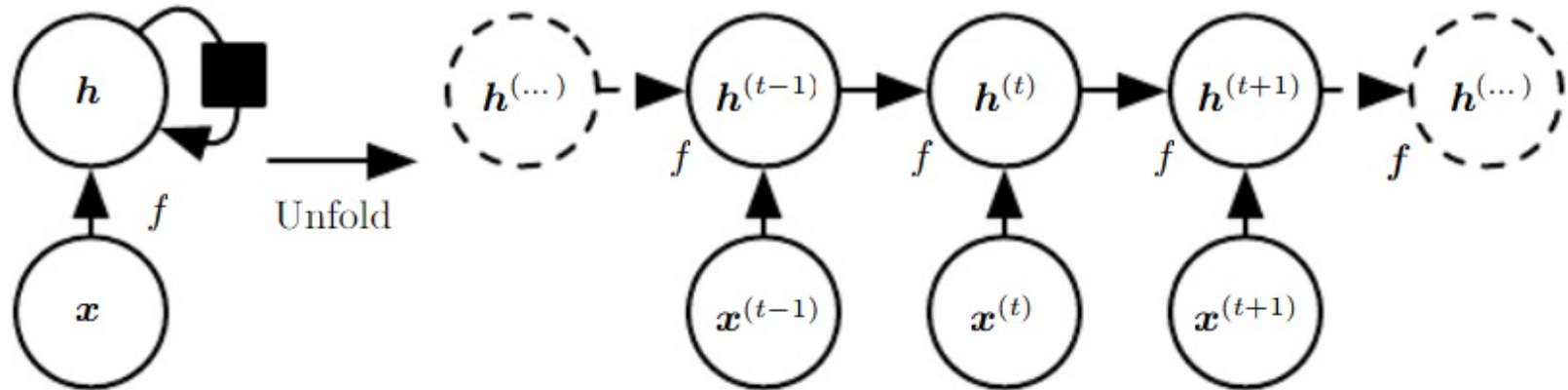
$$\mathbf{O}_t = \mathbf{H}_t \mathbf{W}_{ho} + \mathbf{b}_o.$$

Parameters of the RNN include the weights $\mathbf{W}_{xh} \in \mathbb{R}^{d \times h}$, $\mathbf{W}_{hh} \in \mathbb{R}^{h \times h}$, and the bias $\mathbf{b}_h \in \mathbb{R}^{1 \times h}$ of the hidden layer, together with the weights $\mathbf{W}_{ho} \in \mathbb{R}^{h \times o}$ and the bias $\mathbf{b}_o \in \mathbb{R}^{1 \times o}$ of the output layer, then we have $\mathbf{O}_t \in \mathbb{R}^{n \times o}$.

It is worth mentioning that even at different time steps, RNNs always use these model parameters.

Computational Graph

Computation graphs are graphs with equational data. They are a form of directed graphs that represent a mathematical expression. Plotting computational graphs helps us visualize the dependencies of operators and variables within the calculation.



Backpropagation Through Time

We start with a simplified model of how an RNN works. The mathematical notation here does not explicitly distinguish scalars, vectors, and matrices as it used to do.

$$\begin{aligned}h_t &= g(x_t, h_{t-1}, w_h), \\o_t &= f(h_t, w_o),\end{aligned}$$

where f and g are transformations of the hidden layer and the output layer, respectively. The loss function is denoted as:

$$L(x_1, \dots, x_T, y_1, \dots, y_T, w_h, w_o) = \frac{1}{T} \sum_{t=1}^T l(y_t, o_t).$$

It measures the discrepancy between output o_t and the desired label y_t . By the chain rule,

$$\begin{aligned}\frac{\partial L}{\partial w_h} &= \frac{1}{T} \sum_{t=1}^T \frac{\partial l(y_t, o_t)}{\partial w_h} \\&= \frac{1}{T} \sum_{t=1}^T \frac{\partial l(y_t, o_t)}{\partial o_t} \frac{\partial f(h_t, w_o)}{\partial h_t} \frac{\partial h_t}{\partial w_h}.\end{aligned}$$

The first and the second factors of the product are easy to compute. The third factor is where things get tricky,

Backpropagation Through Time

$$\begin{aligned}h_t &= g(x_t, h_{t-1}, w_h), \\ o_t &= f(h_t, w_o),\end{aligned}$$

According to the recurrent computation, h_t depends on both h_{t-1} and w_h , where computation of h_{t-1} also depends on w_h . Thus, using the chain rule yields:

$$\frac{\partial h_t}{\partial w_h} = \frac{\partial g(x_t, h_{t-1}, w_h)}{\partial w_h} + \frac{\partial g(x_t, h_{t-1}, w_h)}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial w_h}.$$

Lemma

Assume that we have three sequences $\{a_t\}, \{b_t\}, \{c_t\}$ satisfying $a_0 = 0$ and $a_t = b_t + c_t a_{t-1}$ for $t = 1, 2, \dots$. Then it is easy to show

$$a_t = b_t + \sum_{i=1}^{t-1} \left(\prod_{j=i+1}^t c_j \right) b_i.$$

Backpropagation Through Time

By substituting according to:

$$\begin{aligned}a_t &= \frac{\partial h_t}{\partial w_h}, \\b_t &= \frac{\partial g(x_t, h_{t-1}, w_h)}{\partial w_h}, \\c_t &= \frac{\partial g(x_t, h_{t-1}, w_h)}{\partial h_{t-1}},\end{aligned}$$

We can get

$$\frac{\partial h_t}{\partial w_h} = \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h} + \sum_{i=1}^{t-1} \left(\prod_{j=i+1}^t \frac{\partial f(x_j, h_{j-1}, w_h)}{\partial h_{j-1}} \right) \frac{\partial f(x_i, h_{i-1}, w_h)}{\partial w_h}.$$

The Vanishing and Exploding Gradients Problem

$$\frac{\partial L}{\partial W} \propto \sum_{j=0}^T \left(\prod_{i=j+1}^y \frac{\partial h_i}{\partial h_{i-1}} \right) \frac{\partial h_i}{\partial W}$$

This chain can get very long whenever t is large. Two common problems that occur during the backpropagation of time-series data are the vanishing and exploding gradients. The equation above has two problematic cases:

1. *Vanishing gradient* $\left\| \frac{\partial h_i}{\partial h_{i-1}} \right\|_2 < 1$

2. *Exploding gradient* $\left\| \frac{\partial h_i}{\partial h_{i-1}} \right\|_2 > 1$



The Vanishing and Exploding Gradients Problem

The Vanishing and Exploding Gradients Problem

Remember when we do the gradient descent:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W},$$

where η is the step size.

In the first scenario, the term approaches 0 exponentially quickly (so $\frac{\partial L}{\partial W}$ will be close to 0), making it difficult to understand some long-term dependencies. This is known as the *vanishing gradient problem*. In the second situation, the term approaches infinity at an exponential rate, and its value becomes a NaN as a result of the unstable process. This is known as the *exploding gradient problem*.

Dealing with Vanishing Gradients

Challenge Question: How to deal with
vanishing gradients?



RNN Variations

Introduction

When faced with real-life challenges, we may encounter the following scenarios:

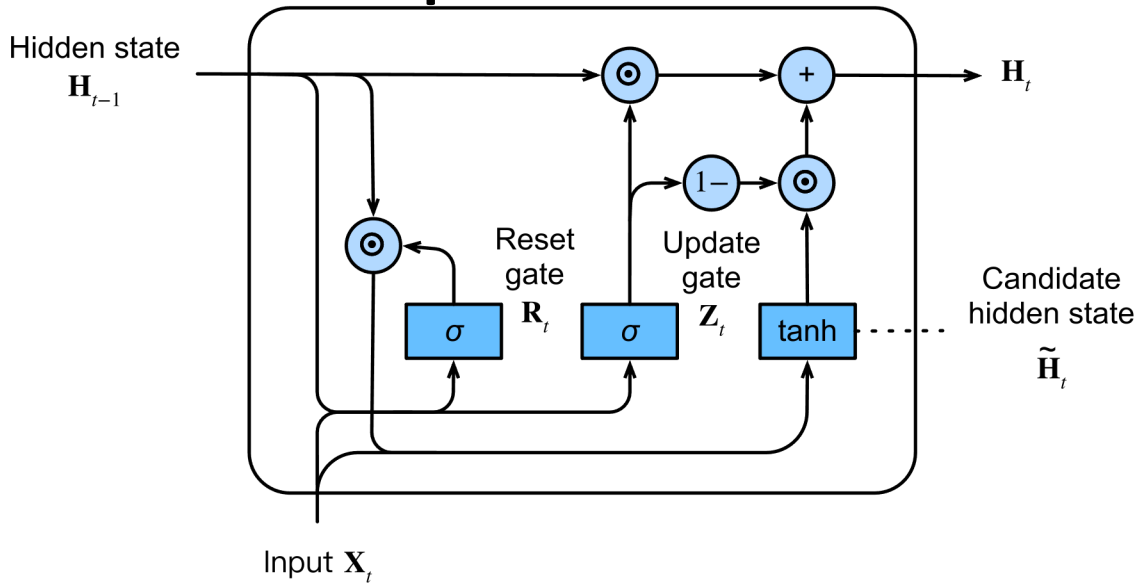
- As we've learned that the gradients may **vanish or explode** during the updating process if the time steps are long.
- We may come across a situation in which an early observation is extremely important in predicting all subsequent observations.
- We may come across situations in which some observations are irrelevant.
- We may come across situations in which there is a logical break between parts of a series. For example, there could be a transition between a bull and bear market in stocks, or between chapters in a book. We'd like to have a method for resetting our internal state representation.

GRU

GRU

The gated recurrent unit (GRU) is one of the RNN variants to overcome the shortcomings of a vanilla RNN. The main difference between vanilla RNNs and GRUs is that the latter allows for hidden state gating. This means that we have separate processes for updating and resetting hidden states. These techniques are learned and handle the issues mentioned above. For example, if the first observation is critical, we will learn not to change the hidden state following the first observation. Similarly, we will learn to disregard irrelevant transitory observations. Finally, we will learn how to reset the latent state as needed. We discuss this in detail below.

Computation Graph



Computing the hidden state in a GRU model

GRU

Let's revisit how a vanilla RNN updates the hidden states. If the hidden layer's activation function is ϕ , and the input is a mini-batch of examples $\mathbf{X} \in \mathbb{R}^{n \times d}$ with batch size n and d inputs, then the hidden states are updated as follows:

$$\mathbf{H}_t = \phi(\mathbf{X}_t \mathbf{W}_{xh} + \mathbf{H}_{t-1} \mathbf{W}_{hh} + \mathbf{b}_h).$$

How GRU does this in a different way?

$$\mathbf{H}_t = \mathbf{Z}_t \odot \mathbf{H}_{t-1} + (1 - \mathbf{Z}_t) \odot \tilde{\mathbf{H}}_t,$$

where $\odot$ is the Hadamard (elementwise) product operator.

Hadamard Product (Element-wise Multiplication)

The name should be sufficiently self-illustrative. Let's look at an example:

The order of matrices/vectors to be multiplied should be the same and the resulting matrix will also be of the same order.

$$\begin{matrix} & G & & & H & & & N \\ \begin{bmatrix} 3 & 5 & 7 \\ 4 & 9 & 8 \end{bmatrix} & \circ & \begin{bmatrix} 1 & 6 & 3 \\ 0 & 2 & 9 \end{bmatrix} & = & \begin{bmatrix} 3 \times 1 & 5 \times 6 & 7 \times 3 \\ 4 \times 0 & 9 \times 2 & 8 \times 9 \end{bmatrix} \end{matrix}$$

GRU

Let's go through this step by step.

$$\mathbf{H}_t = \mathbf{Z}_t \odot \mathbf{H}_{t-1} + (1 - \mathbf{Z}_t) \odot \tilde{\mathbf{H}}_t.$$

$\mathbf{Z}_t$ is called the **update gate**. What we hope is that whenever the update gate $\mathbf{Z}_t$ is near 1, we will simply keep the old state. The information from $\mathbf{X}_t$ is simply ignored in this situation, thereby skipping time step t in the dependency chain. In contrast, when the update gate $\mathbf{Z}_t$ approaches zero, the new latent state approaches the **candidate latent state** (which will be covered in greater depth later). These designs can assist us in dealing with the *vanishing gradient problem* in RNNs and in better capturing dependencies for sequences with large time step distances. For example, if the update gate has been near to 1 for all time steps of a subsequence, the old hidden state at its beginning will be easily kept and passed to its end, regardless of the duration of the subsequence. The way to calculate is similar to how we calculate the hidden state in a vanilla RNN:

$$\mathbf{Z}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xz} + \mathbf{H}_{t-1} \mathbf{W}_{hz} + \mathbf{b}_z),$$

Candidate Hidden State

Finally, let's dive into the **candidate hidden state** $\hat{\mathbf{H}}_t$. As the name implies, a candidate hidden state is a hidden state that we propose but left to the update gate to decide to what extent we use it. It's calculated as follows:

$$\hat{\mathbf{H}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{xh} + (\mathbf{R}_t \odot \mathbf{H}_{t-1}) \mathbf{W}_{hh} + \mathbf{b}_h),$$

Here we use a nonlinearity in the form of $\tanh$ to ensure that the values in the candidate hidden state remain in the interval $(-1, 1)$. Whenever the entries in the reset gate $\mathbf{R}_t$ are close to 1, we recover a vanilla RNN. For all entries of the reset gate $\mathbf{R}_t$ that are close to 0, the candidate hidden state is the result of an MLP with $\mathbf{X}_t$ as input. Any pre-existing hidden state is thus reset to defaults.

Candidate Hidden State

An *update gate* is a gate that determines whether we need to update the hidden state. Similarly, a **reset gate** determines whether we need to reset the hidden state. It is calculated in a similar manner as the update gate:

$$\begin{aligned}\mathbf{Z}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xz} + \mathbf{H}_{t-1} \mathbf{W}_{hz} + \mathbf{b}_z), \\ \mathbf{R}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xr} + \mathbf{H}_{t-1} \mathbf{W}_{hr} + \mathbf{b}_r),\end{aligned}$$

A Complete Review

$$\begin{aligned}\mathbf{R}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xr} + \mathbf{H}_{t-1} \mathbf{W}_{hr} + \mathbf{b}_r), \\ \mathbf{Z}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xz} + \mathbf{H}_{t-1} \mathbf{W}_{hz} + \mathbf{b}_z), \\ \tilde{\mathbf{H}}_t &= \tanh(\mathbf{X}_t \mathbf{W}_{xh} + (\mathbf{R}_t \odot \mathbf{H}_{t-1}) \mathbf{W}_{hh} + \mathbf{b}_h), \\ \mathbf{H}_t &= \mathbf{Z}_t \odot \mathbf{H}_{t-1} + (1 - \mathbf{Z}_t) \odot \tilde{\mathbf{H}}_t.\end{aligned}$$

Let' review. Try to answer the following questions:

What happens when $\mathbf{R}_t$ is close to 1? What happens when $\mathbf{R}_t$ is close to 0?

What happens when $\mathbf{Z}_t$ is close to 1? What happens when $\mathbf{Z}_t$ is close to 0?

What are the purposes of $\mathbf{R}_t$ and $\mathbf{Z}_t$?

Summary

In summary, GRUs have the following two distinguishing features:

- Reset gates help capture short-term dependencies in sequences.
- Update gates help capture long-term dependencies in sequences.

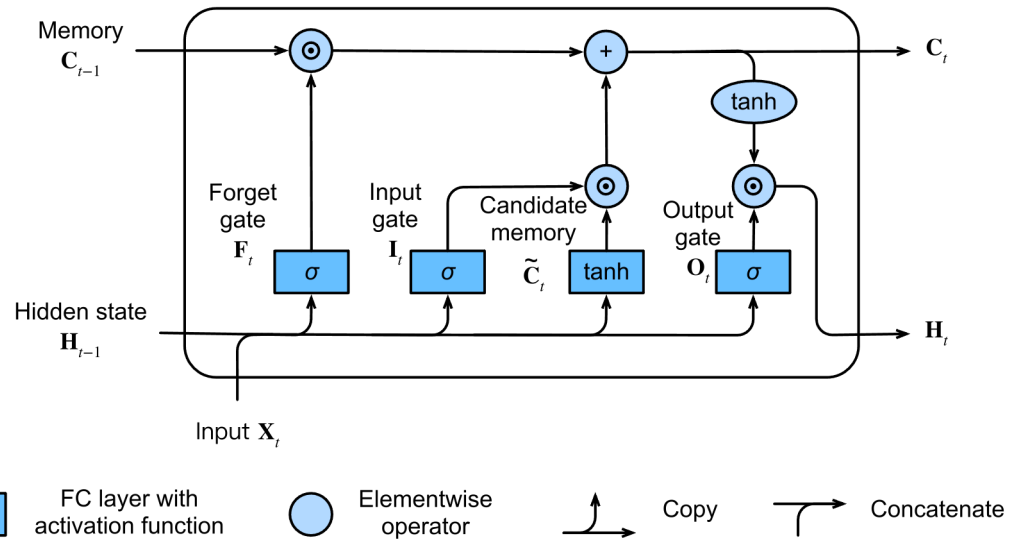
LSTM

LSTM

Long Short-Term Memory (LSTM) is a model with a structure similar to but slightly more sophisticated than GRU. Interestingly, LSTM was invented considerably earlier than GRU, despite being more complicated. Hochreiter and Schmidhuber invented LSTM in 1997, whereas Cho et al. invented GRU in 2014. Therefore, LSTM precedes GRUs by almost two decades.

Summary

- Computing the hidden state in a GRU model



The Calculation of LSTM

The construction of an LSTM:

$$\begin{aligned}\mathbf{I}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xi} + \mathbf{H}_{t-1} \mathbf{W}_{hi} + \mathbf{b}_i), \\ \mathbf{F}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xf} + \mathbf{H}_{t-1} \mathbf{W}_{hf} + \mathbf{b}_f), \\ \mathbf{O}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xo} + \mathbf{H}_{t-1} \mathbf{W}_{ho} + \mathbf{b}_o), \\ \tilde{\mathbf{C}}_t &= \tanh(\mathbf{X}_t \mathbf{W}_{xc} + \mathbf{H}_{t-1} \mathbf{W}_{hc} + \mathbf{b}_c), \\ \mathbf{C}_t &= \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t, \\ \mathbf{H}_t &= \mathbf{O}_t \odot \tanh(\mathbf{C}_t).\end{aligned}$$

The construction of a vanilla RNN:

$$\mathbf{H}_t = \phi(\mathbf{X}_t \mathbf{W}_{xh} + \mathbf{H}_{t-1} \mathbf{W}_{hh} + \mathbf{b}_h).$$

LSTM

The design of the LSTM is arguably inspired by computer logic gates. LSTM introduces a **memory cell** (or cell for short) that is engineered to record additional information and has the same shape as the hidden state (some literature considers the memory cell to be a special type of the hidden state). To control the memory cell we need a number of gates. One gate is needed to read out the entries from the cell. This is referred to as the **output gate**. To determine when to read data into the cell, a second gate is required. This is referred to as the **input gate**. Finally, we require a way to reset the cell's content, which is managed by a **forget gate**. The motivation for such a design is the same as for GRUs: to be able to choose when to recall and when to ignore inputs in the hidden state via a specialized method. Let's see how this plays out in practice.

LSTM

Suppose the input is $\mathbf{X}_t \in \mathbb{R}^{n \times d}$ and the hidden state of the previous time step is $\mathbf{H}_{t-1} \in \mathbb{R}^{n \times h}$, the gates at time step t are defined as follows: the input gate is $\mathbf{I}_t \in \mathbb{R}^{n \times h}$, the forget gate is $\mathbf{F}_t \in \mathbb{R}^{n \times h}$, and the output gate is $\mathbf{O}_t \in \mathbb{R}^{n \times h}$. They are calculated in a very similar manner as a hidden state in a vanilla RNN:

$$\begin{aligned}\mathbf{I}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xi} + \mathbf{H}_{t-1} \mathbf{W}_{hi} + \mathbf{b}_i), \\ \mathbf{F}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xf} + \mathbf{H}_{t-1} \mathbf{W}_{hf} + \mathbf{b}_f), \\ \mathbf{O}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xo} + \mathbf{H}_{t-1} \mathbf{W}_{ho} + \mathbf{b}_o),\end{aligned}$$

where $\mathbf{W}_{xi}, \mathbf{W}_{xf}, \mathbf{W}_{xo} \in \mathbb{R}^{d \times h}$ and $\mathbf{W}_{hi}, \mathbf{W}_{hf}, \mathbf{W}_{ho} \in \mathbb{R}^{h \times h}$ are weight matrices and $\mathbf{b}_i, \mathbf{b}_f, \mathbf{b}_o \in \mathbb{R}^{1 \times h}$ are bias vectors.

DESIGNING MEMORY CELL

Candidate Cell

Next, we design the memory cell. We first introduce the **candidate memory cell**.
What does it look like?

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{xc} + \mathbf{H}_{t-1} \mathbf{W}_{hc} + \mathbf{b}_c),$$

Memory Cell

In GRUs, we have a mechanism to control input and forgetting (or skipping). Similarly, in LSTMs, we have two specialized gates for this purpose: the input gate $\mathbf{I}_t$ controls how much new data we take into account via $\tilde{\mathbf{C}}_t$, and the forget gate $\mathbf{F}_t$ controls how much of the old memory cell content $\mathbf{C}_{t-1} \in \mathbb{R}^{n \times h}$ we keep. We arrive at the following update equation using the same pointwise multiplication approach as before:

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t.$$

Compare it with the calculation of hidden states in GRU:

$$\mathbf{H}_t = \mathbf{Z}_t \odot \mathbf{H}_{t-1} + (1 - \mathbf{Z}_t) \odot \tilde{\mathbf{H}}_t.$$

If the forget gate is always close to 1 and the input gate is always close to 0, the previous memory cells $\mathbf{C}_{t-1}$ are saved over time and passed to the current time step. This system is proposed to address the vanishing gradient problem and to better capture long-term interdependence inside sequences.

Hidden State

Last, we need to define how to compute the hidden state $\mathbf{H}_t \in \mathbb{R}^{n \times h}$. This is where the output gate comes into play. In LSTM it is simply a gated version of the $\mathbf{H}_t$ of the memory cell. This ensures that the values are always in the interval $(-1, 1)$.

$$\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t).$$

When the output gate approaches 1, we effectively pass all memory information to the predictor, whereas when the output gate approaches 0, we maintain all memory information just within the memory cell and perform no further processing. Please notice that this is not the same as *discarding* the information. The information are retained in the memory cell, which can be utilized in a structure like *seq2seq* (which will be covered later).

A Complete Review

The construction of an LSTM:

$$\begin{aligned}\mathbf{I}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xi} + \mathbf{H}_{t-1} \mathbf{W}_{hi} + \mathbf{b}_i), \\ \mathbf{F}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xf} + \mathbf{H}_{t-1} \mathbf{W}_{hf} + \mathbf{b}_f), \\ \mathbf{O}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xo} + \mathbf{H}_{t-1} \mathbf{W}_{ho} + \mathbf{b}_o), \\ \tilde{\mathbf{C}}_t &= \tanh(\mathbf{X}_t \mathbf{W}_{xc} + \mathbf{H}_{t-1} \mathbf{W}_{hc} + \mathbf{b}_c), \\ \mathbf{C}_t &= \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t, \\ \mathbf{H}_t &= \mathbf{O}_t \odot \tanh(\mathbf{C}_t).\end{aligned}$$

Try to answer the following questions:

What happens when $\mathbf{F}_t$ is close to 0 or 1? What about $\mathbf{I}_t$?

What happens when $\mathbf{O}_t$ is close to 0 or 1?

What are the purposes of each of the 3 gates?



THANK YOU!